h e p i a
Haute école du paysage, d'ingénierie
et d'architecture de Genève

Hes·so GENÈVE
Haute Ecole Spécialisée
de Suisse occidentale

FPGA Acceleration of DNA Strand Clustering



Semester project presented by

Adrian Spycher

Computer science and communication systems with orientation in embedded computer systems

Mars, 2025

Teacher HES in charge

Laurent Gantel

Client

**UNIGE, Pierre-Yves Burgi,
Deputy Director at DiSTIC, Head of the
RISe Department**

Caption and source of cover illustration :
from the-scientist.com, ref. URL01

Table of Contents

Summary ..... **V**

List of Acronyms ..... **VI**

List of Figures ..... **VII**

List of Tables ..... **IX**

List of Appendices ..... **X**

Introduction **1**

Chapter 1 : Algorithm Selection and Theoretical Analysis **4**

1.1 Introduction to Levenshtein Distance 4

1.2 Algorithm Choice 5

1.2.a Wagner-Fischer 5

1.2.b Bitap 7

1.2.c BLAST 8

1.2.d Smith-Waterman & Needleman-Wunsch 10

1.2.e Myers' Bit-parallel Algorithm 12

1.3 Choice Justification 14

Chapter 2 : Implementation and Benchmarking in C **16**

2.1 Implementation Details 16

2.1.a Wagner-Fischer 16

2.1.b Myers' Bit-parallel Algorithm 17

2.2 Testing and Benchmarking 17

Chapter 3 : VHDL Implementation and Hardware Optimization **20**

3.1 Architecture Overview 20

3.2 Module Description 21

3.2.a File Loader 21

3.2.b Encoder and Decoder 22

3.2.c Myers Controller 23

3.2.d Myers Core 23

3.2.e Clustering 23

3.3 Core Component of Myers' Algorithm 23

Conclusion 26

Appendix 1 : C Implementation of Wagner-Fischer Algorithm 28

Appendix 2 : C Implementation of Myers' Bit-parallel Algorithm 30

Appendix 3 : VHDL Implementation of Myers' Bit-parallel Algorithm 32

Documentary References 36

Summary

Today, the entire world is accumulating enormous amounts of computer data. This forces the construction of polluting infrastructure and wastes energy. DNAMIC is a European project that aims to store data in synthetic DNA. To achieve this, a microfactory will be created to read and write on synthetic DNA. This microfactory contains six steps: encoding, synthesis, amplification, storage, sequencing, and finally, decoding. After reading the DNA, errors can appear, either via the reading device or due to DNA mutations. To correct these errors, the PCR technique is used to multiply the DNA. With copies of the DNA, it becomes easier to correct errors. This is where sequence assembly, a sub-step of the microfactory, comes into play, as it's necessary to group the copies relative to their original DNA. Grouping these copies requires calculating their Levenshtein distance. A metric that gives the number of operations required to get from one string a to another string b. These operations are the same as those that can occur in DNA mutations: substitution, addition, and deletion. In order to handle the largest possible amount of data, the greatest possible speed is required. The goal of this project is to leverage the speed of FPGAs to accelerate this calculation. The first step in this project is to find the most viable algorithm for this task. This is Myers' bit-parallel algorithm. An initial implementation in C is carried out for testing and familiarization purposes with the algorithm. The next step is to create a VHDL implementation of this algorithm. This has been covered in this project.



Applicant :

Spycher Adrian

Study area : ISC

Teacher in charge :

Laurent Gantel

In collaboration with : UNIGE, DNAMIC

Work subject to a company internship agreement : no

Work subject to a confidentiality agreement : no

List of Acronyms

URL	Uniform Resource Locator
DNA	Deoxyribonucleic Acid
PCR	Polymerase Chain Reaction
FPGA	Field-Programmable Gate Arrays
VHSIC	Very High Speed Integrated Circuit
VHDL	VHSIC Hardware Description Language
RNA	Ribonucleic Acid
BLAST	Basic Local Alignment Search Tool
BLASTn	Basic Local Alignment Search Tool Nucleotide

List of Figures

Figure 1 Schema of the six steps constituting the micro-factory of the DNAMIC project 1
Figure 2 The MinION nanopore sequencer by Oxford Nanopore Technology 2
Figure 3 Wagner-Fischer matrix between “sitting” and “kitten” 7
Figure 4 Example of the bit mask table T of the Bitap algorithm 8
Figure 5 Representation of the BLAST algorithm 9
Figure 6 Scoring method of the Smith–Waterman algorithm 11
Figure 7 Myers’ bit-parallel algorithm dynamic programming (d.p.) matrices 13
Figure 8 Wagner-Fischer matrix with Ukkonen optimization 17
Figure 9 Benchmark between the Wagner-Fischer and Myers’ bit-parallel algorithm implemented in C 19
Figure 10 Block diagram of FPGA architecture 21
Figure 11 Example of DNA strands provided in a text file 22
Figure 12 Example of DNA strand encoding 23
Figure 13 Pipeline architecture of the Myers core component 24

URL References

URL01 https://www.the-scientist.com/making-dna-data-storage-a-reality-30218

URL02 https://www.youtube.com/watch?app=desktop&v=gNv72a1ugiA

URL03 https://nanoporetech.com/products/sequence/minion

URL05 https://en.wikipedia.org/wiki/Wagner%E2%80%93Fischer_algorithm

URL06 https://medium.com/data-science-in-your-pocket/fuzzy-matching-algorithms-explained-e0ff30cc00ca

URL07 https://bio.libretexts.org/Bookshelves/Computational_Biology/Book%3A_Computational_Biology_-_Genomes_Networks_and_Evolution_(Kellis_et_al.)/03%3A_Rapid_Sequence_Alignment_and_Database_Search/3.05%3A_The_BLAST_algorithm_(Basic_Local_Alignment_Search_Tool)

URL08 https://en.wikipedia.org/wiki/Smith%E2%80%93Waterman_algorithm

URL09 https://www.gersteinlab.org/courses/452/09-spring/pdf/Myers.pdf

URL10 https://ceptord.net/wagner-fischer/index.html

List of Tables

Table 1 Different types of edit distance and which operations they allowed 4

Table 2 Comparison of sequence alignment and edit distance algorithms 14

Table 3 Example of synthetic data generated for testing C implementations 18

Table 4 Binary encoding of nucleotides for FPGA processing 22

URL References

URL04 https://en.wikipedia.org/wiki/Edit_distance

List of Appendices

Appendix 1 : C Implementation of Wagner-Fischer Algorithm 28

Appendix 2 : C Implementation of Myers' Bit-parallel Algorithm 30

Appendix 3 : VHDL Implementation of Myers' Bit-parallel Algorithm 32

Introduction

In recent years, the amount of data has continued to increase to the point where, today, we have huge infrastructures dedicated to information storage. These infrastructures are the cause of enormous energy waste. However, it's difficult to eliminate them given that sectors such as education, medicine, and others depend on them. With today's technology, we are now capable of storing data in synthetic (Deoxyribonucleic acid) DNA. This method overcomes many of the problems that come with traditional data storage, especially knowing that all data on earth can be stored in a small test tube.

A European project called DNAMIC aims to replace these infrastructures by a mobile, self-contained, future-proof microfactory for DNA data archiving. This microfactory would only be finished after implementing the six steps that constitute it, that is, encoding, synthesis, amplification, storage, sequencing and finally, decoding.

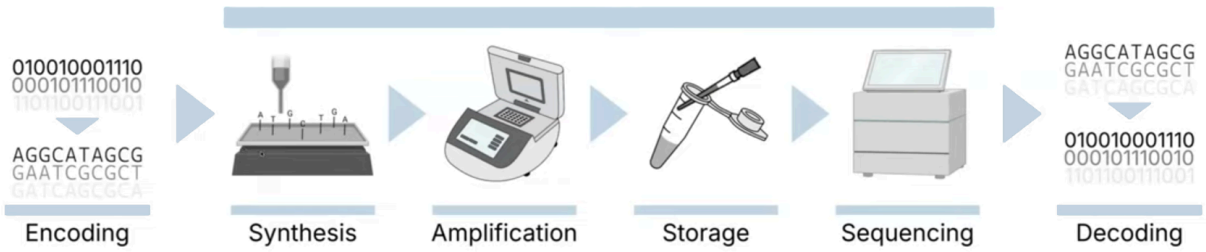


Figure 1: Schema of the six steps constituting the micro-factory of the DNAMIC project

Source *: from youtube.com, ref. URL02*

One of many difficulties this project encounters is located at a sub-step between the sequencing and decoding steps, that we can call the sequence assembly. After storing the synthetic DNA for the necessary time, it must be read. This is done using a nanopore, like the MinION in the Figure 2, a technology that can read 100 to 200 strands of DNA simultaneously at a cost of 10 percent error. In addition to errors due to reading, DNA can also undergo mutations. These errors must then be corrected to maintain data integrity.



Figure 2: The MinION nanopore sequencer by Oxford Nanopore Technology

Source *: from nanoporetech.com, ref. URL03*

To address this issue, engineers at Genomika, a biotechnology company in collaboration with DNAMIC, have chosen to use the PCR technique to generate multiple copies of each DNA strand, enabling error correction in subsequent steps. At this stage of the project, sequence assembly becomes essential. This seemingly small but crucial step is responsible for clustering strands originating from the same initial sequence.

DNA strand clustering is a well-known problem that generates a lot of scientific debate due to its complexity and importance in the field of DNA data storage and sequencing. To achieve clustering, researchers draw parallels with techniques used in spell-checking tools, where differences between two text entries are calculated. This metric can be viewed as a score, where a lower value indicates a better result. Unlike spell-checking tools, where simple differences can be sufficient, DNA strand clustering faces other challenges. Due to the variability introduced throughout the pipeline, DNA strands are not only prone to substitution errors (where one nucleotide is replaced by another) but also to deletion (where one or more nucleotides are missing) and insertion (where extra nucleotides are added). To address these alterations, more advanced distance measures must be employed, in our case, the Levenshtein distance. Today, and especially in our context, the question is no longer how to calculate this metric but how to succeed in calculating it in the shortest possible time. This project aims to support DNAMIC by exploring hardware acceleration techniques with Field-Programmable

Gate Arrays (FPGAs) to optimize the calculation of the Levenshtein distance in order to efficiently manage large volumes. Increasing the number of DNA strands managed by a factor of 100 (between 300 and 500 million) while ensuring reasonable computation times of around a few hours.

The successful implementation of these improvements could have a significant impact on the DNAMIC project and the broader field of DNA data storage. By leveraging FPGA-based acceleration, large-scale DNA clustering will be feasible within practical time constraints. This advancement would enable researchers to process and classify vast datasets without compromising accuracy.

To accomplish this goal, the first step is to make an initial research phase. This phase will focus on investigating and analyzing various algorithms to determine which one is the most suitable for calculating the Levenshtein distance, taking into account the compatibility of the algorithm with hardware acceleration. It will be explained all along the Chapter 1.

The second part of the project involves implementing the most relevant algorithms in C to evaluate their performance. This implementation not only allows for in-depth testing, but also allows for benchmarking on real data. This makes it easier to analyze the effectiveness of algorithms, compare their performance, and identify potential optimizations. It will be discussed in Chapter 2.

The third part discusses the Very High Speed Integrated Circuit (VHSIC) Hardware Description Language (VHDL) implementation of these algorithms. It introduces the general architecture of the project and describes most of the modules. For this semester project, only the core module of the project was implemented and therefore described in detail. Similarly, the section concerning parallelization will be discussed but has not yet been implemented. This information can be found in the Chapter 3.

Chapter 1 : Algorithm Selection and Theoretical Analysis

1.1 Introduction to Levenshtein Distance

In computer science, the edit distance is a metric that measures the difference between two strings. This is done by counting the minimum number of operations needed to transform one string a into another string b. This metric is commonly applied in spelling correction tools, where it helps identify and suggest the closest correct word when a misspelling is detected. Above all, the edit distance plays an important role in bioinformatics, where it is essential for comparing different RNA or DNA sequences. By doing so, it enables researchers to detect, analyze and study mutations.

There are several types of editing distance that can be categorized according to the operations they support and allow. On this table, you can see different types on the Table 1.

Algorithm	**Operations Allowed**			
	Insertions	**Deletions**	**Substitutions**	**Transpositions**
Hamming Distance			✓	
Longest Common Subsequence	✓	✓		
Levenshtein Distance	✓	✓	✓	
Damerau-Levenshtein Distance	✓	✓	✓	✓
Jaro Distance				✓

Table 1: Different types of edit distance and which operations they allowed

Source *: from en.wikipedia.org, ref. URL04*

As mentioned in the introduction, DNA is subject to three types of operations: insertion, deletion and substitution. These are due to the variability introduced throughout the DNA processing pipeline. Only two of these last edit distances achieve the desired result, the Levenshtein distance and the Damerau-Levenshtein distance. These two differ by the fact that the Damerau-Levenshtein include transpositions in addition to the classical edit operations.

An operation that does not appear in the modifications that DNA strands can suffer. Thus, it goes without saying that Levenshtein distance is the most appropriate for our project.

The naive recursive approach to compute the Levenshtein distance is given by the Eq. 1.

$$\mathrm{lev}(a,b) = \begin{cases} n, & \text{if } m = 0 \\ m, & \text{if } n = 0 \\ \mathrm{lev}(\mathrm{tail}(a), \mathrm{tail}(b)), & \text{if } \mathrm{head}(a) = \mathrm{head}(b) \\ 1 + \min \begin{cases} \mathrm{lev}(\mathrm{tail}(a), b), \\ \mathrm{lev}(a, \mathrm{tail}(b)), \\ \mathrm{lev}(\mathrm{tail}(a), \mathrm{tail}(b)), \end{cases} & \text{otherwise} \end{cases} \tag{1}$$

Knowing that n and m are respectively the length of the strings a and b. That the head function return the first character of a string, and that the tail function return the remaining part of a string after removing the first character.

As an example, let's consider the following task : correct the word "kitten" to form the word "sitting". One can easily notice that the Levenshtein distance between these two words is three, meaning it takes three operations to convert one word into the other. Here's a breakdown :

- The first operation is substituting the "k" in "kitten" with an "s", changing "kitten" to "sitten".
- The second operation is substituting the "e" in "sitten" with an "i", transforming "sitten" to "sittin".
- The final operation is inserting an "g" at the end of "sittin", resulting in "sitting".

In total, three operations (two substitutions and one insertion) are required to transform "kitten" into "sitting", which makes the Levenshtein distance between them equal to three.

1.2 Algorithm Choice

a) Wagner-Fischer

The Wagner-Fischer algorithm builds upon the naive approach by incorporating dynamic programming. It works by breaking the problem into smaller sub-problems and solving them

one after the other to retrieve the solution from the original problem. In the case of the Wagner-Fischer algorithm, a matrix is used to store every Levenshtein distance between all prefixes of the string a and b, the last computation being the distance between the two full string. The code below shows a naive pseudo of the Wagner-Fischer algorithm. An improved one is explained on the Section 2.1.a.

```
function wagner_fischer(char a[1..n], char b[1..m]):
 declare int d[0..n, 0..m]

 for i from 1 to n:
    d[i, 0] := i

 for j from 1 to m:
    d[0, j] := j

 for j from 1 to m:
    for i from 1 to n:
       if s[i] = t[j]:
         substitutionCost = 0
       else:
         substitutionCost = 1

       d[i, j] := minimum(d[i-1, j] + 1,              // deletion
                          d[i, j-1] + 1,              // insertion
                          d[i-1, j-1] + substitutionCost)  // substitution

return d[n, m]
```

The matrix in the Figure 3 represents the matrix filled by the Wagner-Fischer algorithm between the words “sitting” and “kitten”. The highlighted path shows the most optimized path for moving from one string to the other. If we want to correct the word “sitting” to “kitten”, we can think of a diagonal movement as a substitution (or no operation if the letters are identical), movements to the right as an insertion, and movements downward as a deletion.

		k	i	t	t	e	n
	0	1	2	3	4	5	6
s	1	1	2	3	4	5	6
i	2	2	1	2	3	4	5
t	3	3	2	1	2	3	4
t	4	4	3	2	1	2	3
i	5	5	4	3	2	2	3
n	6	6	5	4	3	3	2
g	7	7	6	5	4	4	3

Figure 3: Wagner-Fischer matrix between "sitting" and "kitten"

Source *: from en.wikipedia.org, ref. URL05*

One of the key strengths of the Wagner-Fischer algorithm lies in its simplicity. But, unfortunately, when dealing with long strings, this algorithm is not very efficient. With some optimization, Wagner-Fischer has a time complexity of $O(n \times k)$, n being the length of the query (string a) and k the maximum tolerated distance. Another drawback is the memory requirement, as the algorithm needs to store a matrix of size $O(n \times m)$. This can be optimized to $O(n)$ by storing only the current and previous rows of the matrix. Note that the memory will, in any case, become significant when comparing a lot of data simultaneously.

b) Bitap

The Bitap algorithm is one of the most widely used algorithms in search tools, partly because it is the underlying algorithm for the Unix utility *agrep*. Originally, the Bitap algorithm allowed finding an exact pattern P of length n in a text T of length m. It does this efficiently by using bitwise operations to maintain bit vectors that track matches of P as it scans T. With some optimization, it can also perform fuzzy searching, meaning it can find approximate matches of P within T according to a threshold k.

This is done by maintaining multiple bit vectors, each representing the state of the search, while allowing a different number of errors. Instead of tracking a single match state, we define $k + 1$ bit vectors, R_0 through R_k, where R_e represents matches with up to e errors.

To efficiently track character matches, the algorithm precomputes a bitmask table, often referred to as the T table. This table stores, for each character in the alphabet σ, a bit

vector indicating its positions in the pattern P. Instead of scanning P for each character in the text, the algorithm can quickly determine matches using bitwise operations with these precomputed masks.

If we take a pattern P of ababc to be searched in abdababab, the alphabet would be $\sigma = \{a, b, c, d\}$ and the T bit mask table would look like the table in the Figure 4.

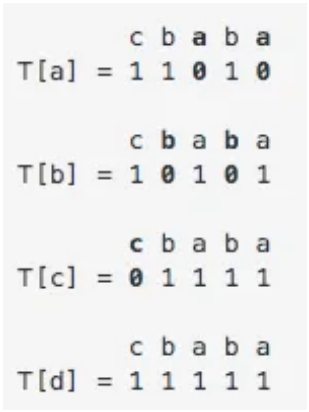


Figure 4: Example of the bit mask table T of the Bitap algorithm

Source *: from medium.com, ref. URL06*

By processing the text from left to right and updating these bit vectors at each step, the algorithm efficiently finds approximate matches, where the last bit of any R_e vector indicates whether a match with at most e errors has been found. Despite its worst-case complexity of $O(k \times n)$, Bitap with errors remains practical for small k due to the efficiency of bitwise operations, making it widely used in search utilities like *agrep* and modern fuzzy search tools. It has a total space complexity of $O(k \times m + \sigma)$. Bitap delivers excellent practical performance, as it manages to do most of the work with bitwise operations, which are extremely fast. It goes without saying that using binary operations is a major advantage when it comes to implementing the algorithm on FPGAs.

c) BLAST

The Basic Local Alignment Search Tool (BLAST) is one of the most popular algorithms for comparing biological sequences, particularly DNA, RNA, and protein sequences. It is a local sequence alignment, meaning it finds the most similar sub-sequences between two strings

rather than aligning the entire sequences. BLAST works by breaking down a query sequence into small sub-sequences known as words. It then searches in a database for exact or near-exact matches to these words. Once matches are found, BLAST extends them in both directions to determine the length and quality of the alignment. If the extended alignment surpasses a predetermined score threshold k, it is reported as a match.

One of the key components of BLAST's efficiency is its use of a hash table. It stores all possible sub-sequences of letters, called words. A word is of a specific length w typically between three or eleven letters. The size of the hash table depends on the alphabet used: for nucleotide sequences, there are 4^w possible words, while for protein sequences, there are 20^w possible words. This table significantly speeds up the search process, as it allows for efficient lookups of potential matches in the database. For example, suppose that the DNA sequence contains the following stretch of nucleotides: AGTCCG. If a BLAST search is conducted under normal conditions with a word size of three nucleotides, the words in the hash table would be AGT, GTC, TCC, and CCG.

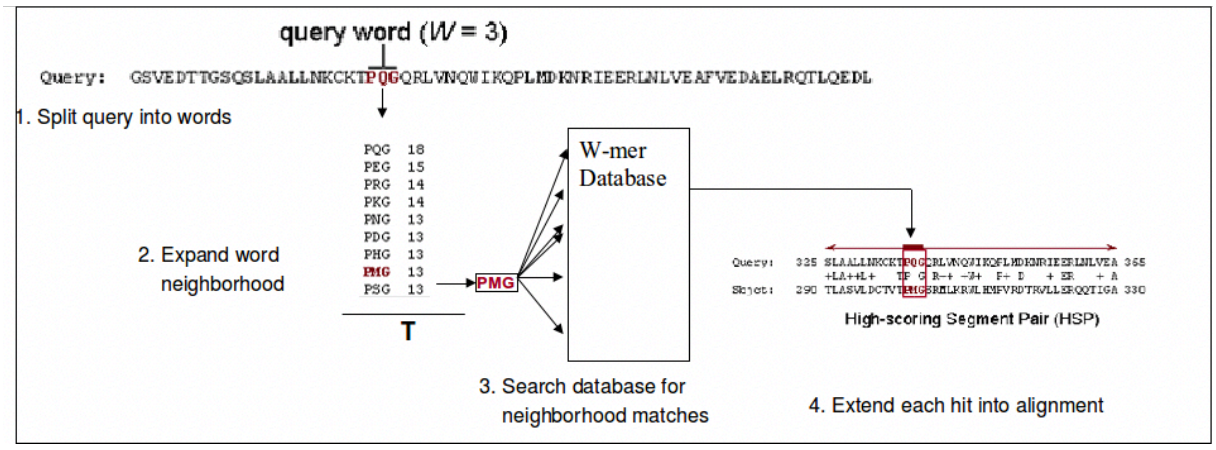


Figure 5: Representation of the BLAST algorithm

Source *: from bio.libretexts.org, ref. URL07*

The time complexity of BLAST is generally $O(D)$, where D is the size of the database. This makes BLAST much faster than exact string matching algorithms, as it does not compare every possible alignment, but instead quickly identifies regions likely to be similar. However, it adds the risk that some optimal alignments may be missed.

In terms of space complexity, BLAST requires memory for the hash table, which contains 4^w entries for nucleotide sequences, as well as for the alignments that are extended during the search. The total space complexity is thus $O(4^w + D)$, which is slightly higher than other alignment algorithms, but still efficient compared to more computationally expensive methods like dynamic programming.

BLAST comes in several variations, each using specific types of data. The most suitable algorithm for this project is Basic Local Alignment Search Tool Nucleotide (BLASTn), which compares nucleotide sequences. Despite its efficiency, the complex memory requirements make its implementation on FPGAs challenging.

d) Smith-Waterman & Needleman-Wunsch

Unlike BLAST, Smith-Waterman is not heuristic, which makes it more interesting depending on the context of the project, as in our case. A bit like Wagner-Fischer, it works by using a matrix where each cell contains a score representing the best possible alignment between prefixes of the two sequences. The scoring is based on matches, mismatches, and gaps (insertions or deletions).

It works by filling a matrix, comparing characters from both sequences, and choosing the optimal score at each step. The score represents the probability of a good alignment. The higher the score, the better the alignment of the two sub-strings. Smith-Waterman fills the matrix iteratively using a substitution matrix, often described as an s function. A trivial s function could be as described in Eq. 2.

$$s(a_i, b_j) = \begin{cases} +1, & a_i = b_j \\ -1, & a_i \neq b_j \end{cases} \tag{2}$$

The cell being processed takes the best score between:

1. The diagonal cell plus the match/mismatch score (representing a substitution).
2. The cell above minus a gap penalty W_k(representing a deletion).
3. The cell to the left minus a gap penalty W_l (representing an insertion).

4. Zero, to indicate that starting a new alignment at this position is better than extending a poor one.

After filling the matrix, the algorithm traces back from the highest scoring cell to reconstruct the optimal local alignment.

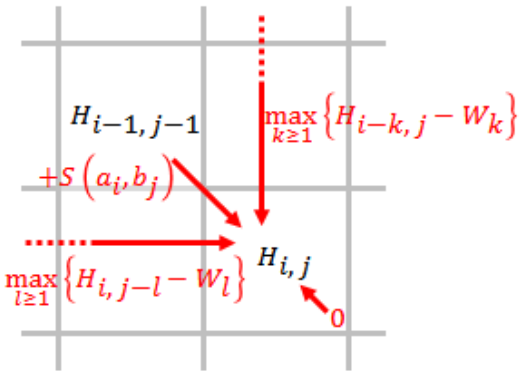


Figure 6: Scoring method of the Smith–Waterman algorithm

Source *: from en.wikipedia.org, ref. URL08*

With Gotoh and Altschul's optimizations, the Smith-Waterman algorithm has a time complexity of $O(n \times m)$. The cost of this greater complexity than BLAST is due to the fact that this algorithm is not heuristic. However, there is a problem for our context; the algorithm performs local alignment when a global one is needed. But there's a trick to fix this: the Needleman-Wunsch algorithm.

The Needleman-Wunsch algorithm is very similar to the Smith-Waterman algorithm. It differs in a few ways, but the most important is that these scores are potentially negative. This is one of the main reasons why this algorithm is capable of global alignment. Like Smith-Waterman, Needleman-Wunsch has a complexity of $O(n \times m)$. Unfortunately, it also requires a traceback at the end of the algorithm to find the alignment. Tracing back this algorithm is not a simple matter in VHDL. It is therefore possible but difficult to achieve hardware acceleration on this algorithm.

e) Myers' Bit-parallel Algorithm

Myers' Bit-parallel algorithm is a highly efficient method for edit-distance-based string matching. Unlike traditional dynamic programming approaches (e.g., Wagner-Fischer), which rely on filling a two-dimensional matrix, Myers' algorithm optimizes this process using bit-parallelism (a bit like Bitap). Myers' bit-parallel algorithm represents the state of the computation using bit vectors. Each bit vector compactly encodes whether a match, substitution, insertion, or deletion is needed at each position in the pattern. By leveraging bitwise operations, the algorithm efficiently simulates the dynamic programming table updates.

To better understand this concept, let's look at how Myers went from computing an $n \times m$ matrix to computing five bit vectors in parallel: Eq, Pv, Ph, Mv, and Mh. To do this, we need to understand several key principles derived from the traditional n by m matrix. In the Wagner-Fischer algorithm, when optimizing memory space, we note that a column C_j of the matrix can be computed using only the column C_{j-1} that precedes it. A related observation is that the difference between adjacent entries in any row or column is either $+1$, 0, or -1. There is a more general way to present this, found in the 4-Russians algorithm for string comparison. Formally, we have the *horizontal delta* $\Delta h[i, j]$ as $C[i, j] - C[i, j-1]$ and the *vertical delta* $\Delta v[i, j]$ as $C[i, j] - C[i-1, j]$ for all $(i, j) \in [1, m] \times [1, n]$. Δv and Δh are matrices that represent the traditional Wagner-Fischer matrix, but relative to the preceding cells, vertically or horizontally. Using several theorems, Myers manages to transform the problem of calculating the Wagner-Fischer matrix into the calculation of the matrix Δv except for the last step where Δh is necessary. But this can be calculated using a simple product on a part of Δv.

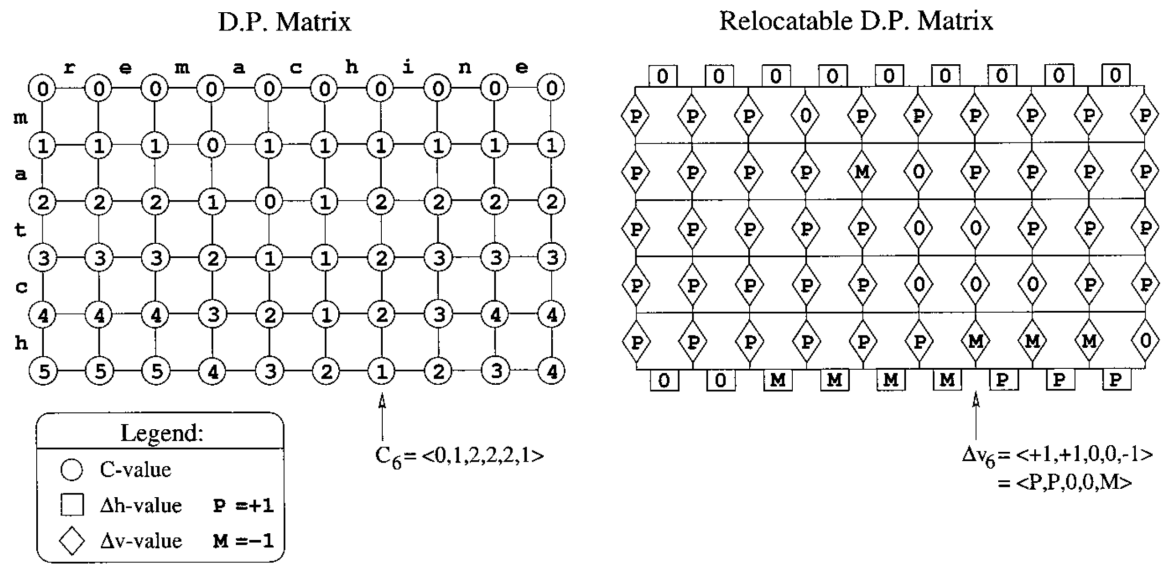


Figure 7: Myers' bit-parallel algorithm dynamic programming (d.p.) matrices

Source *: from gersteinlab.org, ref. URL09*

The Figure 7 shows, on the left, the traditional matrix found in Wagner-Fischer and, on the right, the Δv matrix and the first and last rows of the Δh matrix. The illustration shows that the data in the Δv matrix are Boolean variables M or P. They represent whether an addition (P) or subtraction (M) of 1 occurs on a cell. They cannot both be true (be equal to 1) at the same time. To represent a row, simply create a bit vector, hence the bit vectors Pv, Ph, Mv, and Mh, where v and h represent the axes where the operation is performed (vertical or horizontal). To correctly calculate these bit vectors, it is necessary to know whether the current characters are similar or not. Eq, a bit vector, is responsible for doing so. It represents equality between the characters. Eq can be calculated with a simple comparison, but to save time on the computation of a column of $O(1)$, it is clever to precompile it into a matrix named $Peq[\sigma]$, with σ being the alphabet over which the two strings originate.

```
precompute Peq
Pv = 1^m
Mv = 0
score = m
for j from 1 to n:

  Eq = Peq[j]
```

```
Xv = Eq | Mv
Xh = (((Xv & Pv) + Pv) ^ Pv) | Xv

if lsb(Ph) then score++
if lsb(Mh) then score--

Ph = Pv & Xh
Mh = Mv | ~(Pv | Xh)
Pv = (Mh << 1) | ~0
Mv = (Ph << 1)
```

Myers' bit-parallel algorithm has a very good time complexity of $O\left(n + \frac{m}{w}\right)$, with w being the length of a machine word, and a space complexity of $O(n)$. The use of binary operations helps in several ways. It significantly speeds up computations since these operations can generally be performed in a single clock cycle. Furthermore, it greatly simplifies the hardware acceleration.

1.3 Choice Justification

The algorithms presented in the previous section are all interesting, but they each come with their own strengths and weaknesses. To make the following easier, Table 2 summarizes their main features.

Algorithms	Time Complexity	Space complexity	Range	Suitability
Wagner-Fischer	$O(k \times n)$	$O(n)$	global	good
Bitap	$O(k \times n)$	$O(k \times m + \sigma)$	global	high
BLAST	$O(D)$	$O(4^w + n \times m)$	local	bad
Smith-Waterman	$O(n \times m)$	$O(n)$	local	bad
Needleman-Wunsch	$O(n \times m)$	$O(n)$	global	bad
Myers' Bit-parallel	$O\left(n \times \frac{m}{w}\right)$	$O(n)$	global	high

Table 2: Comparison of sequence alignment and edit distance algorithms

Source *: by Adrian Spycher*

A very important eliminatory criteria in the context of this project is the algorithm's range. Indeed, algorithms with a local range are not suitable for this project, as they do

not allow for the evaluation of overall similarity between sequences. They focus only on identifying the most similar subsequences, which can lead to fragmented results that are not representative of the entire sequence. Therefore, the BLAST and Smith-Waterman algorithms cannot be used for this project.

Regarding the second most important criteria, time complexity, Myers' bit-parallel algorithm wins hands down with a theoretical speed of $\mathrm{O}\left(n \times \frac{m}{w}\right)$, as w can be chosen as the same value of m in FPGA. In order to compare this speed with another algorithm, it was necessary to choose another algorithm to implement. Among the remaining algorithms, although Needleman-Wunsch and Bitap were good candidates, Wagner-Fischer was implemented. This is because it was the project's original proposed algorithm. And in order to see how Myers' bit-parallel algorithm optimized the original proposed algorithm, it was chosen.

Chapter 2 : Implementation and Benchmarking in C

It is important to start by understanding and seeing a C implementation of the Wagner-Fischer and Myers' bit-parallel algorithm. First, it allows familiarization with the implemented algorithm, while also providing a better understanding of the data dependencies and linked operations that are important during the VHDL implementation. This also allows comparing the speed of these two algorithms on the DNA inputs available in the project. These implementations also created a database, allowing for future testing of the VHDL implementation.

2.1 Implementation Details

The Wagner-Fischer and Myers' bit-parallel algorithm implementations are taken from a study by Fujimoto Seiji, author of Polyleven, a Python library used to calculate the Levenshtein distance. In this 2020 study, Fujimoto Seiji implements both algorithms in order to compare them, as in this project. The implementations below are therefore based on these implementations.

a) Wagner-Fischer

The Wagner-Fischer implementation includes several optimizations. The first is a simple optimization of the space that uses the algorithm. Indeed, in this algorithm, the task is to calculate an $n \times m$ matrix, more specifically, the cell located at the bottom right of the matrix. However, since only the column preceding the one being calculated is required to advance through the matrix, it is not necessary to store the entire matrix. With this technique, the algorithm goes from using a memory space of $n \times m$ to a size of n.

A second optimization is the Ukkonen optimization. It reduces the complexity from $\mathrm{O}(n \times m)$ to $\mathrm{O}(n \times k)$, where k is the maximum tolerated distance. It works by excluding certain cells from the table that are sure to be above k. Normally, cells in the upper-right or lower-left corner. Making the algorithm compute at most $n(2k + 1)$ cells. This is well illustrated in the Figure 8.

		p	q	r	s
	0	1	*	*	*
a	1	1	2	*	*
b	*	2	2	3	*
c	*	*	3	3	4
d	*	*	*	4	4

Figure 8: Wagner-Fischer matrix with Ukkonen optimization

Source *: from ceptord.net, ref. URL10*

On the example between “abdc” and “pqrs” in Figure 8, we can see that the Ukkonen optimization removes the computation of 12 cells. So we only need to fill the remaining 13 cells to compute the distance of Levenshtein between the two strings. The implementation with both optimizations is available in Appendix 1.

b) Myers’ Bit-parallel Algorithm

Myers’ bit-parallel algorithm has remained unchanged, so there isn’t much to add. In its implementation, available in Appendix 2, we can still notice the use of variables of type uint256_t. Indeed, the DNA strands used are character strings of more than 200 characters and must be represented by bit vectors of the same length, i.e., uint256_t. These caused a problem because, given that a machine word generally does not exceed 64 bits, it is difficult for the *gcc-13* compiler to handle them. However, *gcc-14* offers an extension to the C standard that provides greater flexibility in handling custom-sized integers. This extension allows the representation of integers of arbitrary width. The creation of uint256_t was therefore possible using the code below.

```
typedef unsigned _BitInt(256) uint256_t;
```

Since the machine used doesn’t run *gcc-14* by default, it was necessary to create a Docker container. This approach simplifies the setup process.

2.2 Testing and Benchmarking

In order to test the two C implementations, a database containing words and their Levenshtein distance is, unfortunately, as necessary as it is difficult to find. So creating one was more conve-

nient. To populate the database, a Python script was developed to generate synthetic data. This script creates two random character strings and calculates their corresponding Levenshtein distance, providing the necessary data for the database. The first part of this database is filled with the Faker library, which provides random words from a chosen language. The second part is filled with false DNA strands (*ACGT*) of variable length. To calculate their Levenshtein distances, the Python library "Levenshtein" was used. This reliable library is open-source and used by over 11,000 users. An example of the data in Table 3.

Text	Pattern	Levenshtein Distance
sometimes	she	7
kelm	hello	3
CTGACCAG ... ATACGTTG	GTTAGCCG ... GCGGACCT	153
TCATTTGC ... CCGTTAGA	CAGACGGA ... AAGCGACT	93

Table 3: Example of synthetic data generated for testing C implementations

Source *: by Adrian Spycher*

The test program is a very naive C code. It retrieves the data and prepares it so it can be used by the two algorithms. It then feeds it to the algorithm, testing its results against the results contained in the data. If an error occurs, it reports it. With more than 1′000 database entries, the programs didn't notice even one error. Both implementation were correct.

The benchmark is a simple C code that calculates the time it takes for the Myers' bit-parallel algorithm and the Wagner-Fischer algorithm to compare n data points. The data it compares are fixed-length strings containing random characters from the following list: "0123456789abcdef".[1] The first benchmark launched tested the time it took the algorithms to compare eight datasets of different sizes: 64, 128, 256, 1024, 2048, 4096, 8192 and 32728. This benchmark was clearly enough to know which algorithm was the most efficient; the data are regrouped on the Figure 9.

[1] A text with only "ACGT" does not change the result.

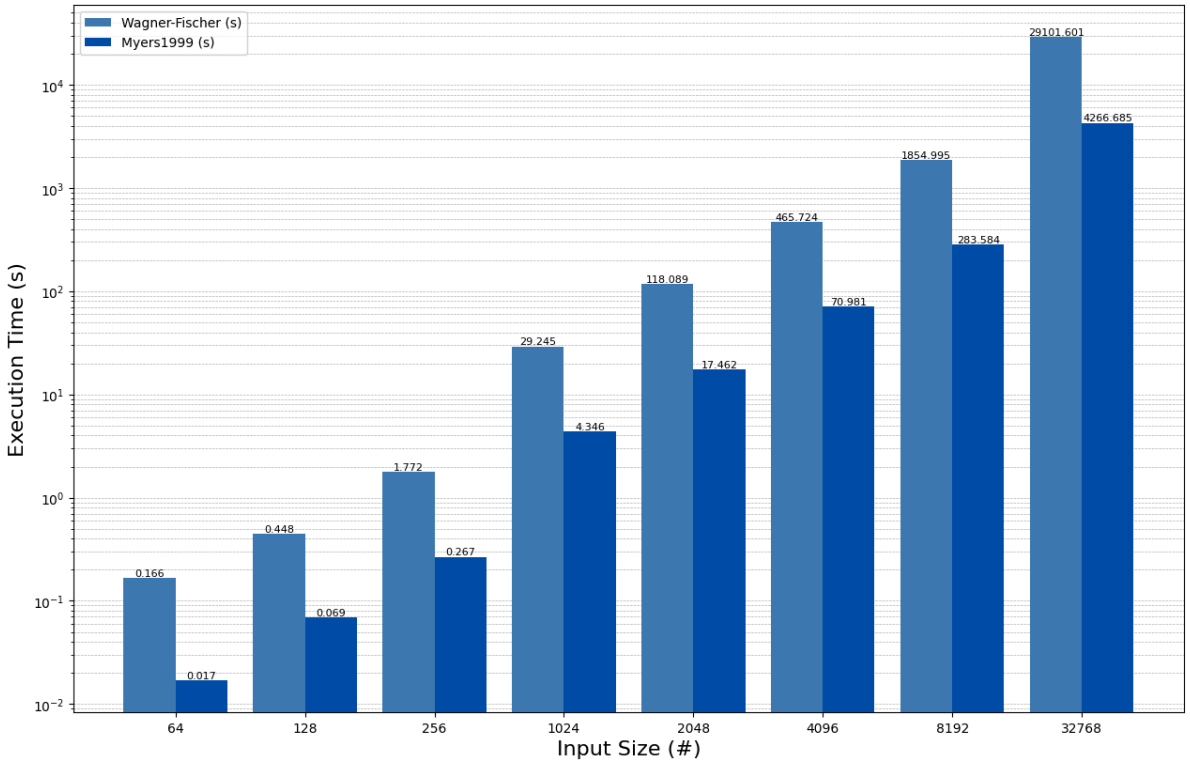


Figure 9: Benchmark between the Wagner-Fischer and Myers' bit-parallel algorithm implemented in C

Source *: by Adrian Spycher*

The graph in Figure 9 shows the execution time in seconds of the two algorithms to compare a given number of inputs. Although the spacing between the Wagner-Fischer and Myers bars appears identical across groups of inputs, the scale is actually logarithmic. This means that for a number of 32,768, there is a factor of 6.82 between Wagner-Fischer and Myers. This places Myers far ahead of Wagner-Fischer in terms of speed, which was expected given their time complexity.

Chapter 3 : VHDL Implementation and Hardware Optimization

Implementing a Levenshtein distance calculation algorithm in VHDL presents several challenges. Indeed, these types of algorithms are very resource-intensive. In an FPGA, this translates into a limited number of LUTs. Even after thoroughly understanding the algorithm and implementing it in C, translating it into VHDL remains a challenging task, as C and VHDL are two very different languages, where certain methods and logic do not work in both. However, programs exist for translating them, but they are not used in this project. To maximize the speed of the Myers algorithm, it was preferable to do it manually in order to have complete control over the VHDL implementation.

The greatest added value offered by VHDL is the parallelization it provides, but it is also one of the biggest challenges of this project. Performing multiple comparisons simultaneously allows the computation to be divided across multiple logical units, thus optimizing execution time. Furthermore, the use of pipelining allows processing steps to be chained together without waiting for previous calculations to complete, thus improving throughput and reducing overall system latency.

3.1 Architecture Overview

The project architecture is divided into seven modules, each with its own responsibilities. A general diagram can be found in Figure 10.

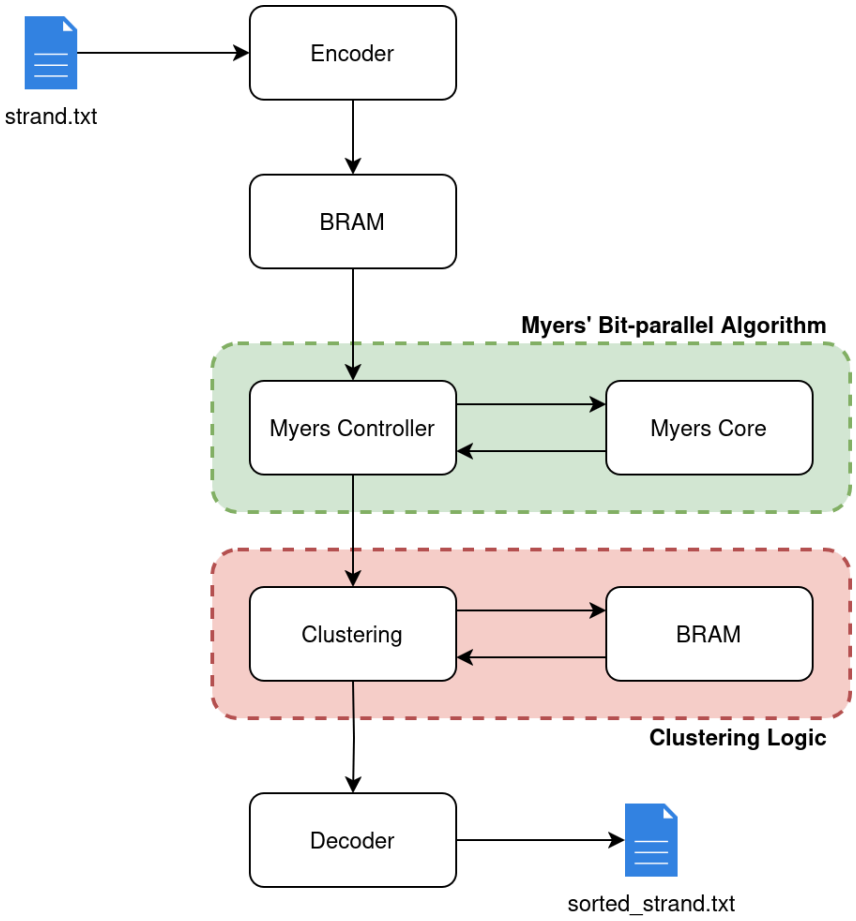


Figure 10: Block diagram of FPGA architecture

Source *: by Adrian Spycher*

Each module communicates via well-defined interfaces, allowing for efficient and parallel execution of different tasks. Some modules operate in a pipelining mode, allowing for continuous data processing and better overall throughput. The architecture was also designed to ensure good scalability, as it might be changed during the bachelor's project.

3.2 Module Description

a) File Loader

The file loader is responsible for reading the input file. This file contains all the DNA strands to be sorted. In other words, the ones to be used to calculate the Levenshtein distance. In the received file, each line corresponds to a DNA strand, as represented in Figure 11.

ATCTGATGTTACGCGCGACATAATCGCCGAGAAGTGC [...]
CAGTCCAAGGCACCGGTAATAGCGAGATGACTGGAG
ATCTGATGTTACGCGCGATAGTAGACACCGCGCCGTT [...]
TAGAGAGTACTGAGCGACATAGCGAGATGACTGGAG
ATCTGATGTTACGCGCGACGACTACGTAGATCTATGT [...]
TACCTGTATCTGGAGATCATAGCGAGATGACTGGAG
ATCTGATGTTACGCGCGACTCAGACTGCGTTCGGTAA [...]
CACCGTAGACAGATCTGTATAGCGAGATGACTGGAG
ATCTGATGTTACGCGCGAGCTCGAGCGACGTCTGCTA [...]
CGTCCTCGTTGCTGCAACATAGCGAGATGACTGGAG
...

Figure 11: Example of DNA strands provided in a text file

Source *: by Adrian Spycher*

This module is possible in VHDL using file management libraries such as std or ieee. As output, this module provides the inputs to the encoder as an array of text. It's important to note that this module can only be used in simulation. In a real-life scenario, the data connection would be made using a PCI connection, but this only appears later in the project.

b) Encoder and Decoder

This encoder is a combinational circuit that converts an array containing all text entries representing DNA strands into an array with the same strands but encoded using two bits. It is used to reduce data complexity by assigning a unique code to each active entry.

Nucleotide	Encoding
A	00
C	01
G	10
T	11

Table 4: Binary encoding of nucleotides for FPGA processing

Source *: by Adrian Spycher*

Working with bits allows for faster computations and, more importantly, reduces the memory space needed. A four-character alphabet lets each character be represented by two bits, as shown in the encoding in the Table 4. A conversion example is available in Figure 12.

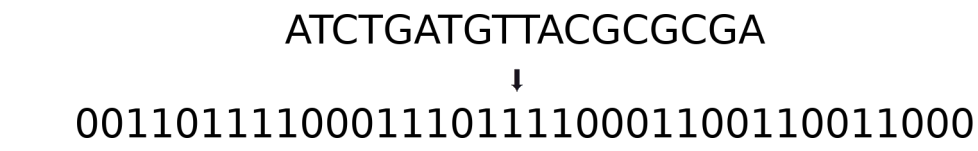


Figure 12: Example of DNA strand encoding

Source *: by Adrian Spycher*

DNAMIC uses a different encoding called CODEC. It is not used in this case because it is designed to maintain data integrity. In our case, the two-bit encoding is designed for hardware use.

c) Myers Controller

The Myers controller is only used to manage the Myers Core module. It manages the inputs and outputs provided to the Myers core component, as well as the various timings that must be respected. In the second part of this project, it will also handle the parallelization of the Myers core module.

d) Myers Core

Myers Core is the heart of the Myers algorithm. It handles the fundamental Myers calculations, those included in the algorithm loop seen in the Section 1.2.e. It is detailed in the Section 3.3.

e) Clustering

The Clustering module assembles DNA strands based on their Levenshtein distance. It groups together the strands closest to each other; this creates groups in which we assume that each strand comes from the same original strand. To store these groups, it uses BRAM, which offers fast dual-port access. When all the groups have been evaluated, it will provide the sorted strands to the decoder.

3.3 Core Component of Myers' Algorithm

As stated previously, the Myers core component plays a very important role in the system. It handles the core calculation of the Myers algorithm. This consists of managing the evolution of the bit vectors as the algorithm progresses until the final score is obtained, i.e., the Levenshtein

distance between two strands. To achieve this, the Myers core is composed of two separate blocks: one for updating the bit vectors, while the other for updating the score. The reason for separating the component into two blocks is due to the fact that there is a dependency between the calculation of the score and two of the bit vectors. Those bit vectors must be stable in order to ensure that the score is correctly computed. To ensure this and maintain a clock frequency as high as possible, a pipelined architecture was developed.

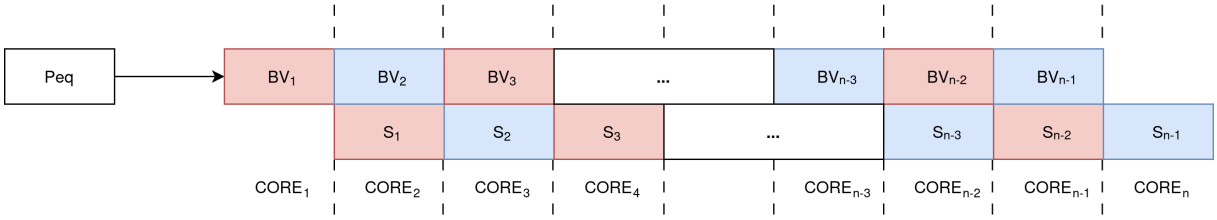


Figure 13: Pipeline architecture of the Myers core component

Source *: by Adrian Spycher*

The diagram in Figure 13 shows the Myers Core component pipeline. The BV_i cells represent the bit vector processing part, while those with S_i cells handle the score. Cells with the same color work for the same cycle of the Myers algorithm. As demonstrated in this figure, the Myers Core components are chained together to obtain the Levenshtein distance at the last cell. Thus, after the first m cycles, we begin to retrieve the Levenshtein distance from the data at each cycle, allowing us to process iteratively. To manage the two separate blocks, the VHDL separates them in two processes, respectively, proc_core and proc_score available in the Appendix 3.

In normal operation, this process updates the vectors according to the current index idx_i, which is used to traverse the sequence. If idx_i is not equal to 1, shift operations are applied to ph and mh to simulate progress in the alignment. Then, the new vectors pv and mv are calculated based on the previous states and the results of the peq_i operation, which stores information about the character matches between the sequences s1_i and s2_i. Finally, the ph_o and mh_o values are updated to reflect the new vector states, and the score update signals (add_score_o, sub_score_o) are calculated based on the last element of the ph_o and mh_o vector.

The proc_score process is responsible for managing the evolution of the edit score based on the operations performed by the proc_core process. Each clock cycle, this process checks whether either the add_score_i or sub_score_i signals are active. If add_score_i is 1, then score_o is incremented by 1. Conversely, if sub_score_i is active, score_o is decremented by 1.

Conclusion

This project involved hardware acceleration of a Levenshtein distance calculation algorithm between two strings, in this case DNA strands. After researching and gaining a thorough understanding of the different existing algorithms, the algorithms were compared. The most suitable algorithm for this project was the Myers algorithm. It is one of the fastest Levenshtein distance calculation algorithms, and, more importantly, it is very well-suited for FPGAs. For testing purposes, it was implemented in C alongside one of its competitors, the Wagner-Fischer algorithm. Their tests concluded that the Myers algorithm was indeed the most suitable for this project. Following this, a review and initial VHDL implementation were undertaken. This review focused on the overall architecture of the project and the circuit components. The most important component of all was implemented first, namely the Myers core component. This expected behavior was explained in the last section of this document.

During this project, there were many challenges to overcome. One of them, which was certainly the most challenging, was the research on alignment algorithms and Levenshtein distance calculations. Indeed, algorithms are often not well-defined; they are rather a base upon which many scientists add their own grain of sand. Whether optimizations for time complexity, space complexity, or modifications to adapt the algorithm to other purposes, it is difficult at first glance to obtain clear and precise information. This information is often subject to change during the research. Even during the writing of these last chapters, new optimizations were found on some of the algorithms.

The bachelor's project will aim to accelerate a crucial step in the DNAMIC project's microfactory. Accelerating the calculation of the Levenshtein distance will allow for the management of much larger amounts of data, providing a vision for the future of this microfactory. This semester project allowed us to learn about and evaluate the possibilities of this optimization. This was a necessary step that ensures the project is moving in the best possible direction.

The VHDL implementation of the Myers algorithm remains a prototype for the time being, and will be used for the bachelor's project. This approach provides a good start to the problem but does not take into account certain issues that will arise when scaling the system. Such as managing synchronization between different processing units, optimizing software and hardware resources, and improving error handling at large scales. Furthermore, aspects related to computational parallelization and efficient memory usage will need to be considered to ensure optimal performance in more complex environments.

Appendix 1 : C Implementation of Wagner-Fischer Algorithm

```
int32_t wagner_fischer(char *s1, int32_t len1, char *s2, int32_t len2) {

    int32_t Mx, mx, dia, top, left;
    int32_t i, j, m, M;
    int16_t buf[4096];
    char *p1;

    if (len1 < len2)
        return wagner_fischer(s2, len2, s1, len1);
    if (len2 == 0)
        return len1;
    if (len2 == 1)
        return len1 - (memchr(s1, s2[0], len1) != NULL);
    if (len2 == 2) {
        p1 = memchr(s1, s2[0], len1 - 1);
        if (p1) {
            i = len1 + s1 - p1 - 1;
            return len1 - (memchr(p1 + 1, s2[1], i) != NULL) - 1;
        } else {
            return len1 - (memchr(s1 + 1, s2[1], len1 - 1) != NULL);
        }
    }

    Mx = (len2 - 1) / 2;
    mx = 1 - Mx - (len1 - len2);

    for (j = 0; j <= Mx; j++)
        buf[j] = j;

    for (i = 1; i <= len1; i++) {
        buf[0] = i - 1;

        m = MAX(mx, 1);
        M = MIN(Mx, len2);
        mx++;
        Mx++;

        dia = buf[m - 1];
        top = buf[m];
```

```
        if (s1[i - 1] != s2[m - 1])
            dia = MIN(dia, top) + 1;

        buf[m] = dia;
        left = dia;
        dia = top;

        for (j = m + 1; j <= M; j++) {
            top = buf[j];

            if (s1[i - 1] != s2[j - 1])
                dia = MIN(MIN(dia, top), left) + 1;
            buf[j] = dia;
            left = dia;
            dia = top;
        }

        if (len2 == M)
            continue;

        if (s1[i - 1] != s2[M])
            dia = MIN(dia, left) + 1;

        buf[M + 1] = dia;
    }

    return buf[len2];
}
```

Appendix 2 : C Implementation of Myers' Bit-parallel Algorithm

```
int32_t myers1999(char *s1, int32_t len1, char *s2, int32_t len2) {

    uint256_t Peq[256];
    uint256_t Eq, Xv, Xh, Ph, Mh, Pv, Mv, Last;
    int32_t i;
    int32_t Score = len2;

    memset(Peq, 0, sizeof(Peq));

    for (i = 0; i < len2; i++)
        Peq[(uint8_t)s2[i]] |= (uint256_t)1 << i;

    Mv = 0;
    Pv = (uint256_t)-1;
    Last = (uint256_t)1 << (len2 - 1);

    for (i = 0; i < len1; i++) {
        Eq = Peq[(uint8_t)s1[i]];

        Xv = Eq | Mv;
        Xh = (((Eq & Pv) + Pv) ^ Pv) | Eq;

        Ph = Mv | ~(Xh | Pv);
        Mh = Pv & Xh;

        if (Ph & Last)
            Score++;
        if (Mh & Last)
            Score--;

        Ph = (Ph << 1) | 1;
        Mh = (Mh << 1);

        Pv = Mh | ~(Xv | Ph);
        Mv = Ph & Xv;
    }
```

```
    return Score;
}
```

Appendix 3 : VHDL Implementation of Myers' Bit-parallel Algorithm

```
library myers;
use myers.myers_pkg.all;

library ieee;
use ieee.numeric_std.all;
use ieee.std_logic_1164.all;
use ieee.std_logic_textio.all;


entity myers_core is

  port (
    clk_i      : in std_logic;
    rst_i      : in std_logic;

    s1_i       : in strand;
    s2_i       : in strand;
    Peq_i      : in peq_array;

    Ph_i       : in strand_vector;
    Pv_i       : in strand_vector;
    Mh_i       : in strand_vector;
    Mv_i       : in strand_vector;
    Xv_i       : in strand_vector;
    idx_i      : in integer;          -- change to generic

    add_score_i : in std_logic;
    sub_score_i : in std_logic;
    score_i    : in integer;

    s1_o       : out strand;
    s2_o       : out strand;
    Peq_o      : out peq_array;

    Ph_o       : out strand_vector;
    Pv_o       : out strand_vector;
    Mh_o       : out strand_vector;
```

```
        Mv_o       : out strand_vector;
        Xv_o       : out strand_vector;

        add_score_o : out std_logic;
        sub_score_o : out std_logic;
        score_o     : out integer
    );

end myers_core;


architecture Behavioral of myers_core is

    ---------------------------------------------------------------------------
    -- signals
    ---------------------------------------------------------------------------

    signal eq, ph, pv, mh, mv, xh : strand_vector;

begin

    ---------------------------------------------------------------------------
    -- core
    ---------------------------------------------------------------------------

    proc_core : process(clk_i)
    begin

        if rising_edge(clk_i) then

            if rst_i = '1' then

                eq        <= (others => '0');
                ph        <= (others => '0');
                pv        <= (others => '0');
                mh        <= (others => '0');
                mv        <= (others => '0');
                xh        <= (others => '0');

                ph_o      <= (others => '0');
                pv_o      <= (others => '0');
                mh_o      <= (others => '0');
```

```
        mv_o       <= (others => '0');
        xv_o       <= (others => '0');

        add_score_o <= '0';
        sub_score_o <= '0';

    else

        if idx_i /= 1 then

            -- shift ph and mh
            ph <= (std_logic_vector(shift_left(unsigned(ph), 1)) and '1');
            mh <= (std_logic_vector(shift_left(unsigned(mh), 1)) and '0');

            -- update pv and mv
            pv <= mh or (not (xv_i or ph));
            mv <= ph and xv_i;
        else

            pv <= pv_i;
            mv <= mv_i;
        end if;

        -- get eq and idx
        eq <= peq_i(to_integer(unsigned(s1_i(idx_i))));

        -- compute xv and xh
        xv_o <= eq or mv_i;
        xh <= std_logic_vector(
            (((unsigned(eq) and unsigned(pv)) + unsigned(pv)) xor unsigned(pv)) or unsigned(eq)
        );

        -- compute new ph and mh
        ph_o <= mv or (not (xh or pv));
        mh_o <= pv and xh;

        -- try to update score instantly ? no pipeline
        -- update score
        add_score_o <= ph_o(strand_len - 1);
        sub_score_o <= mh_o(strand_len - 1);
    end if;
```

```
    end if;
  end process proc_core;

  ---------------------------------------------------------------------------
  -- score
  ---------------------------------------------------------------------------

  proc_score : process(clk_i)
  begin

    if rising_edge(clk_i) then

      if rst_i = '1' then
        score_o <= strand_len;

      else

        if add_score_i = '1' then
          score_o <= score_i + 1;
        end if;

        if sub_score_i = '1' then
          score_o <= score_i - 1;
        end if;

      end if;
    end if;

  end process proc_score;

end behavioral;
```

Documentary References

3.5: The BLAST algorithm (Basic Local Alignment Search Tool), [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://bio.libretexts.org/Bookshelves/Computational_Biology/Book%3A_Computational_Biology_-_Genomes_Networks_and_Evolution_(Kellis_et_al.)/03%3A_Rapid_Sequence_Alignment_and_Database_Search/3.05%3A_The_BLAST_algorithm_(Basic_Local_Alignment_Search_Tool)

B, Chloe, 2025. String search with mechanical sympathy: the Bitap algorithm. [en ligne]. 6 March 2025. [Consulté le 28 March 2025]. Disponible à l'adresse : https://medium.com/@chloebre/string-search-with-mechanical-sympathy-the-bitap-algorithm-8483d8c351dd

Bitap algorithm, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://en.wikipedia.org/w/index.php?title=Bitap_algorithm&oldid=1271757439

BLAST (biotechnology), [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://en.wikipedia.org/w/index.php?title=BLAST_(biotechnology)&oldid=1277187940

ceptord.net, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://www.ceptord.net/index.html

DNA, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://en.wikipedia.org/w/index.php?title=DNA&oldid=1282390679

Edit distance, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://en.wikipedia.org/w/index.php?title=Edit_distance&oldid=1259445915

Faker: Faker is a Python package that generates fake data for you., [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://github.com/joke2k/faker

GADDAM, Yasaswini, 2024. Wagner–Fischer algorithm: Minimum Edit Distance. [en ligne]. 20 April 2024. [Consulté le 28 March 2025]. Disponible à l'adresse : https://medium.com/@yasaswini.gaddam21/wagner-fischer-algorithm-minimum-edit-distance-4e61bba9b656

GUPTA, Mehul, 2024. Fuzzy matching algorithms explained. [en ligne]. 1 February 2024. [Consulté le 28 March 2025]. Disponible à l'adresse : https://medium.com/data-science-in-your-pocket/fuzzy-matching-algorithms-explained-e0ff30cc00ca

Hamming distance, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://en.wikipedia.org/w/index.php?title=Hamming_distance&oldid=1275773583

Haskell, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://en.wikipedia.org/w/index.php?title=Haskell&oldid=1281033176

Levenshtein distance, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://en.wikipedia.org/w/index.php?title=Levenshtein_distance&oldid=1279737312

MYERS, Gene, [sans date]. A Fast Bit-Vector Algorithm for Approximate String Matching Based on Dynamic Programming.

Needleman–Wunsch algorithm, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://en.wikipedia.org/w/index.php?title=Needleman%E2%80%93Wunsch_algorithm&oldid=1259432631

Nucleotide, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://en.wikipedia.org/w/index.php?title=Nucleotide&oldid=1272341271

PEARL-DNA, 2024. State of the Art DNA Data Storage | DigNA Webinar Series (Webinar 1). [en ligne]. 2024. [Consulté le 28 March 2025]. Disponible à l'adresse : https://www.youtube.com/watch?app=desktop&v=gNv72a1ugiA

Polymerase chain reaction, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://en.wikipedia.org/w/index.php?title=Polymerase_chain_reaction&oldid=1281648665

Programmation dynamique, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://fr.wikipedia.org/w/index.php?title=Programmation_dynamique&oldid=216302582

rapidfuzz/Levenshtein, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://github.com/rapidfuzz/Levenshtein

Sequence alignment, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://en.wikipedia.org/w/index.php?title=Sequence_alignment&oldid=1267781271

Smith–Waterman algorithm, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://en.wikipedia.org/w/index.php?title=Smith%E2%80%93Waterman_algorithm&oldid=1281063949

Taxicab geometry, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://en.wikipedia.org/w/index.php?title=Taxicab_geometry&oldid=1281078664

The Myers Difference Algorithm, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://www.nathaniel.ai/myers-diff/

Time complexity, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://en.wikipedia.org/w/index.php?title=Time_complexity&oldid=1282069508

WAGNER, Robert A. and FISCHER, Michael J., 1974. The String-to-String Correction Problem. *Journal of the ACM* [en ligne]. January 1974. Vol. 21, n° 1, pp. 168–173. [Consulté le 28 March 2025]. DOI 10.1145/321796.321811. Disponible à l'adresse : https://dl.acm.org/doi/10.1145/321796.321811

Wagner–Fischer algorithm, [sans date]. [en ligne]. [Consulté le 28 March 2025]. Disponible à l'adresse : https://en.wikipedia.org/w/index.php?title=Wagner%E2%80%93Fischer_algorithm&oldid=1211775080